# ASSIGNMENT 2

# DSA0114 - OBJECT ORIENTED PROGRAMMING WITH C++

Name: Yallareddy Kaluvai

Reg.no: 192465047

# Q1 – Call by Value, by Reference and by Pointer

## Hostel Fee Concession Updater

### Prediction before running (fee = 45000, 10% concession)

| Version | Predicted stored fee | Reason |
|---|---|---|
| By value | 45000 | Function works on a copy; the copy is destroyed on return. |
| By reference | 40500 | The parameter is another name for the caller's variable. |
| By pointer | 40500 | *fee modifies the object the pointer points to. |

### Three implementations (three_ways.cpp)

```
// Q1 - Hostel Fee Concession Updater: by value, by reference, by pointer, plus return-new-fee
#include <iostream>
#include <string>

void byValue(long long fee, int percent)        { fee -= fee * percent / 100;
   std::cout << "  inside byValue    : " << fee << '\n'; }
void byReference(long long& fee, int percent)   { fee -= fee * percent / 100;
   std::cout << "  inside byReference: " << fee << '\n'; }
void byPointer(long long* fee, int percent)     { *fee -= *fee * percent / 100;
   std::cout << "  inside byPointer  : " << *fee << '\n'; }
long long returnNew(long long fee, int percent) { return fee - fee * percent / 100; }

struct Student { std::string name; long long fee; char bigData[1000]; };
void printConst(const Student& s) { std::cout << s.name << " pays " << s.fee << '\n'; }

int main()
{
   long long f1 = 45000, f2 = 45000, f3 = 45000;
   byValue(f1, 10);      std::cout << "stored fee after byValue    : " << f1 << '\n';
   byReference(f2, 10);  std::cout << "stored fee after byReference: " << f2 << '\n';
   byPointer(&f3, 10);   std::cout << "stored fee after byPointer  : " << f3 << '\n';

   std::cout << "\nSeveral concessions (merit 10%, sports 15%, single parent 25%):\n";
   long long fee = 45000;
   fee = returnNew(fee, 10); fee = returnNew(fee, 15); fee = returnNew(fee, 25);
   std::cout << "chained return-new-fee result: " << fee << '\n';
   long long fee2 = 45000;
   byReference(fee2, 10); byReference(fee2, 15); byReference(fee2, 25);
   std::cout << "in-place by reference result : " << fee2 << '\n';

   std::cout << "\nsizeof(long long)=" << sizeof(long long)
             << "  sizeof(Student)=" << sizeof(Student)
             << "  sizeof(Student*)=" << sizeof(Student*) << '\n';
   Student s{"Asha", 40500, {0}};
```

```
  printConst(s);
}
```

```
$ cd Q1
$ g++ -std=c++17 -Wall -Wextra three_ways.cpp -o three && ./three
  inside byValue    : 40500
stored fee after byValue    : 45000
  inside byReference: 40500
stored fee after byReference: 40500
  inside byPointer  : 40500
stored fee after byPointer  : 40500

Several concessions (merit 10%, sports 15%, single parent 25%):
chained return-new-fee result: 25819
  inside byReference: 40500
  inside byReference: 34425
  inside byReference: 25819
in-place by reference result : 25819

sizeof(long long)=8  sizeof(Student)=1040  sizeof(Student*)=8
Asha pays 40500
```

Observed: the value version prints 40500 inside the function but the stored fee stays 45000; reference and pointer versions change the stored fee to 40500. The prediction was correct.

### Inquiry

- **Where each parameter lives:** By value – a new local long long (8 bytes) on the callee's stack frame holding a copy. By pointer – a new local pointer (8 bytes) on the stack holding the caller's address; the data itself stays in the caller. By reference – no new object; the name is an alias of the caller's variable (the compiler usually passes its address internally, but semantically there is only one object).
- **Why only two change the caller's fee:** only the reference and the pointer give the function access to the caller's own object. The value version modifies its private copy, which is why the result printed inside the function looked correct while the stored fee never changed.
- **When to use const reference:** when a function only reads a large object (for example sizeof(Student) = 1040 bytes in my test). const T& avoids the copy and the compiler rejects any accidental modification. For small types such as int or long long, passing by value is just as cheap.

### Design Decision – several concessions

I would return the new fee: long long applyConcession(long long fee, int percent), and the caller writes fee = applyConcession(fee, 10). Reasons: (1) the effect is visible at the call site, so the original bug (a changed copy that is silently thrown away) cannot occur; (2) calls chain naturally for merit, sports and single-parent concessions; (3) the function has no side effects and is easy to test. In-place change by reference is preferable only when the object is large and copying would be costly. Both gave 25819 for 10% + 15% + 25% on 45000 (45000 → 40500 → 34425 → 25819 with integer truncation). Note that the concessions compound; adding them (50%) would give 22500, so the policy must be agreed with the hostel office.

### Predict the Output

```
#include <iostream>
void change(int a, int& b, int* c)
{a += 10;
b += 10;
*c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
int a = 1, b = 2, c = 3;
change(a, b, &c);
```

```
std::cout << a << ' ' << b << ' ' << c << '\n';
bigger(a, c) = 100;
std::cout << a << ' ' << c << '\n';
show(1);
show(1, 2);
show(1, 2, 3);
std::cout << '\n';
}
```

**My prediction (written before running):** line 1: 1 12 13  line 2: 1 100  line 3: 16 13 6 (with a trailing space, then newline).

```
$ cd Q1
$ g++ -std=c++17 -Wall -Wextra predict.cpp -o predict && ./predict
1 12 13
1 100
16 13 6
```

Observed output is identical to the prediction – no differences. Explanation:

- change(a, b, &c): a is a copy, so main's a stays 1; b is a reference, so 2 becomes 12; c is changed through the pointer, so 3 becomes 13.
- bigger(a, c) returns int& to the larger variable. a = 1 and c = 13, so it returns a reference to c, and the assignment = 100 sets c to 100 (a stays 1).
- show(1) uses both defaults y = 5 and z = 10 → 16. show(1, 2) uses only the default z = 10 → 13. show(1, 2, 3) uses no default → 6. Each call prints a space after the number, so the line ends with '6 ' before the newline.

### Complete the Code (TODO 1)

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
   // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by reference)
   fee -= fee * percent / 100;
}
int main()
{
   long long hostelFee = 45000;
   applyByValue(hostelFee, 10);
   std::cout << "after call by value : " << hostelFee << '\n';
   applyConcession(hostelFee, 10);
   std::cout << "after call by reference: " << hostelFee << '\n';
}
```

```
$ cd Q1
$ g++ -std=c++17 -Wall -Wextra complete.cpp -o complete && ./complete
after call by value : 45000
after call by reference: 40500
```

The call-by-value version leaves 45000 because fee is a copy of hostelFee; fee -= ... changes only that copy, which is destroyed when the function returns. The reference version changes hostelFee itself: 45000 − 45000×10/100 = 45000 − 4500 = 40500.

### Alternative Concept and Complexity Analysis

Alternative: long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; } with fee = applyConcession(fee, 10);. Time is O(1) in all versions (one multiplication, one division, one subtraction). The difference is the data copied per call:

| Parameter type | By value | By reference | By const reference |
|---|---|---|---|
| long long (8 B) | 8 B copy | no copy (address only) | no copy (address only) |
| Student (1040 B measured) | 1040 B copy + string copy | no copy | no copy, read-only |
| Return of new long long | 8 B (usually in a register) | – | – |

Space: O(1) extra for the numeric case; for a large Student, by value costs O(size of Student) per call while references cost O(1).

# Q2 – Return by Reference and Its Dangers

## Examination Marks Register

### Implementation (register.cpp)

```
// Q2 - Examination Marks Register: return by reference
#include <iostream>
#include <vector>

int& highest(int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
const int& highestConst(const int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
int highestCopy(const int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return marks[idx];
}
int findIndex(const int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return idx;
}
int* findPointer(int marks[], int n)
{
    int idx = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[idx]) idx = i;
    return &marks[idx];
}
int& dangerous()                        // BAD: reference to a local variable
{
    int local = 99;
    return local;
}
int main()
{
    int marks[5] = {62, 88, 75, 91, 54};
    highest(marks, 5) += 2;                 // call on the left of an assignment operator
    std::cout << "after +2 moderation: ";
    for (int m : marks) std::cout << m << ' ';
    std::cout << '\n';

    int copy = highestCopy(marks, 5); copy += 2;
    std::cout << "highest after copy+2: " << highestCopy(marks, 5) << " (copy was " << copy << ")\n";
    // highestConst(marks,5) += 2;  // compile error: assignment of read-only location
    std::cout << "const reference read: " << highestConst(marks, 5) << '\n';
```

```
    marks[findIndex(marks, 5)] += 2;
    *findPointer(marks, 5) += 2;
    std::cout << "after index and pointer versions: " << highestCopy(marks, 5) << '\n';

    std::vector<int> v = {62, 88, 75, 91, 54};
    int* before = &v[3];
    std::cout << "vector capacity before: " << v.capacity() << ", address of v[3]: " << (void*)before << '\n';
    for (int i = 0; i < 20; ++i) v.push_back(i);
    int* after = &v[3];
    std::cout << "vector capacity after : " << v.capacity() << ", address of v[3]: " << (void*)after << '\n';
    std::cout << (before == after ? "same address" : "ADDRESS CHANGED -> old reference/pointer is dangling") << '\n';

    (void)dangerous;   // never called: using the result would be undefined behaviour
}
```

```
$ cd Q2
$ g++ -std=c++17 -Wall -Wextra register.cpp -o reg && ./reg
register.cpp: In function ‘int& dangerous()’:
register.cpp:38:12: warning: reference to local variable ‘local’ returned [-Wreturn-local-addr
   38 |     return local;
      |            ^~~~~
register.cpp:37:9: note: declared here
   37 |     int local = 99;
      |         ^~~~~
after +2 moderation: 62 88 75 93 54
highest after copy+2: 93 (copy was 95)
const reference read: 93
after index and pointer versions: 97
vector capacity before: 5, address of v[3]: 0x561e5e8bd2cc
vector capacity after : 40, address of v[3]: 0x561e5e8bd37c
ADDRESS CHANGED -> old reference/pointer is dangling
```

### What the program shows

- highest(marks, 5) += 2; changes the array element directly: 91 became 93. The call is on the left of an assignment operator because it returns int& (an lvalue).
- The compiler warned: warning: reference to local variable 'local' returned [-Wreturn-local-addr] (see screenshot). local is destroyed when dangerous() returns, so the returned reference refers to a dead stack slot. Using it is undefined behaviour: it may print 99, garbage, or crash. That is why dangerous() is never called in the program.

| Return type | Meaning | Effect in this program |
|---|---|---|
| int& | Alias of the element; can be modified | highest(...) += 2 changes the array |
| const int& | Alias, read-only; no copy | highestConst(...) += 2 is a compile error; reading is fine |
| int (copy) | Independent value | copy + 2 gave 95 but the array stayed 93 |

### Changed Condition – the register is a growing std::vector

A reference (or pointer) to an element is really the element's memory address. When push_back exceeds the capacity, the vector allocates a bigger block, copies the elements and frees the old block. In my run the capacity went from 5 to 40 and the address of v[3] changed from ...2cc to ...37c. A reference or pointer obtained before the growth now points to freed memory (dangling) and using it is undefined behaviour. The safe pattern is to fetch the reference again after any operation that can grow the vector, to store an index, or to reserve() enough capacity in advance.

### Alternative Concept and Complexity Analysis

```
// index version                    // pointer version
int i = findIndex(marks, n);            int* p = findPointer(marks, n);
marks[i] += 2;                       *p += 2;
// reference version
highest(marks, n) += 2;
```

Time: all three versions scan the array once, O(n) (n − 1 comparisons). Space: O(1) – one index, one pointer or one reference. Safety when the array becomes a growing vector:

| Return | Calling code | Safety after vector growth |
|---|---|---|
| int& | highest(v) += 2; | Unsafe if kept across push_back; fine if used immediately |
| int* | *findPointer(v) += 2; | Same danger; also can be null or be used after free |
| int (index) | v[findIndex(v)] += 2; | Safe – an index stays valid after reallocation (as long as no elements are erased or reordered) |

# Q3 – Default Arguments and Function Overloading

## Courier Charge Calculator

### The compile error (bad.cpp)

```
$ cd Q3
$ g++ -std=c++17 -Wall -Wextra bad.cpp -o bad
bad.cpp: In function ‘int main()’:
bad.cpp:6:33: error: call of overloaded ‘charge(double)’ is ambiguous
    6 | int main() { std::cout << charge(2.0) << ' ' << charge(2.0, true) << '\n'; }
      |                           ~~~~~~^~~~
bad.cpp:2:8: note: candidate: ‘double charge(double)’
    2 | double charge(double weight) { return weight * 40; }
      |        ^~~~~~
bad.cpp:4:8: note: candidate: ‘double charge(double, bool, bool)’
    4 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
bad.cpp:6:55: error: call of overloaded ‘charge(double, bool)’ is ambiguous
    6 | int main() { std::cout << charge(2.0) << ' ' << charge(2.0, true) << '\n'; }
      |                                                 ~~~~~~^~~~~~~~~~
bad.cpp:3:8: note: candidate: ‘double charge(double, bool)’
    3 | double charge(double weight, bool express) { return weight * 40 + (express ? 50 : 0);
      |        ^~~~~~
bad.cpp:4:8: note: candidate: ‘double charge(double, bool, bool)’
    4 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

charge(2.0) matches charge(double) exactly and also matches charge(double, bool = false, bool = false) exactly, because the defaults fill in the missing arguments. Both are exact matches, neither is better, so the compiler reports 'call of overloaded charge(double) is ambiguous'. The same happens for charge(2.0, true) between the two-parameter version and the three-parameter version with a default. The fix is to give each version a distinct parameter list in the first parameter and to keep only one function per kind of input, with all optional arguments as defaults of that one function.

### Redesigned functions (courier.cpp)

```
// Q3 - Courier Charge Calculator: default arguments + overloading, no ambiguity
#include <iostream>

struct Parcel { double kg; bool fragile; };

double extras(bool express, bool insured) { return (express ? 50 : 0) + (insured ? 20 : 0); }

double charge(int grams, bool express = false, bool insured = false)       // (1) grams
{ std::cout << "[int grams] ";  return grams / 1000.0 * 40 + extras(express, insured); }
double charge(double kg, bool express = false, bool insured = false)       // (2) kg
```

```
{ std::cout << "[double kg] ";  return kg * 40 + extras(express, insured); }
double charge(const Parcel& p, bool express = false, bool insured = false) // (3) Parcel
{ std::cout << "[Parcel] ";     return p.kg * 40 + (p.fragile ? 30 : 0) + extras(express, insured); }

int main()
{
   Parcel box{3.0, true};
   std::cout << "Call 1 charge(2500)           -> "; std::cout << charge(2500) << '\n';
   std::cout << "Call 2 charge(2.5)            -> "; std::cout << charge(2.5) << '\n';
   std::cout << "Call 3 charge(2.5, true)      -> "; std::cout << charge(2.5, true) << '\n';
   std::cout << "Call 4 charge(2.5, true, true) -> "; std::cout << charge(2.5, true, true) << '\n';
   std::cout << "Call 5 charge(box)            -> "; std::cout << charge(box) << '\n';
   std::cout << "Call 6 charge(2)              -> "; std::cout << charge(2) << '\n';
   std::cout << "Call 7 charge(2.0f)           -> "; std::cout << charge(2.0f) << '\n';
}
```

**Predictions before execution**

| Call | Predicted overload | Reason / value |
|---|---|---|
| charge(2500) | int grams | exact match int; 2.5 kg × 40 = 100 |
| charge(2.5) | double kg | exact match double; 100 |
| charge(2.5, true) | double kg | express +50 → 150 |
| charge(2.5, true, true) | double kg | +50 +20 → 170 |
| charge(box) | Parcel | 3 × 40 + fragile 30 = 150 |
| charge(2) | int grams | exact match int – this is 2 grams, not 2 kg: 0.08 |
| charge(2.0f) | double kg | float→double is a promotion; float→int is a conversion; promotion wins; 80 |

```
$ cd Q3
$ g++ -std=c++17 -Wall -Wextra courier.cpp -o courier && ./courier
Call 1 charge(2500)           -> [int grams] 100
Call 2 charge(2.5)            -> [double kg] 100
Call 3 charge(2.5, true)      -> [double kg] 150
Call 4 charge(2.5, true, true) -> [double kg] 170
Call 5 charge(box)            -> [Parcel] 150
Call 6 charge(2)              -> [int grams] 0.08
Call 7 charge(2.0f)           -> [double kg] 80
```

Observed output matches every prediction. A design warning from this test: charge(2) means 2 grams while charge(2.0) means 2 kg; a caller who forgets the decimal point gets a very different result. For a real counter I would prefer distinct names or a small Weight type.

**Inquiry**

- **How the compiler chooses:** it collects the candidate functions with the same name, keeps those that can accept the arguments (using defaults for missing ones), then ranks each argument conversion: exact match (including adding const or reference binding) beats promotion (float→double, short/char→int, bool→int), which beats standard conversion (int→double, double→int, float→int). The function that is better for at least one argument and not worse for any is chosen; if there is no unique best, the call is ambiguous.
- **Why defaults go from the right:** arguments are matched to parameters by position from the left, so a call can only leave out trailing arguments. void f(int a = 1, int b) would give no way to omit a while supplying b, so the compiler requires every parameter after a defaulted one to also have a default.

**Alternative Concept and Complexity Analysis**

```
struct ChargeRequest { double kg = 0; bool express = false, insured = false, fragile = false; };
double charge(const ChargeRequest& r) {
   return r.kg * 40 + (r.fragile ? 30 : 0) + (r.express ? 50 : 0) + (r.insured ? 20 : 0);
}
```

```
// template version for any numeric weight type
template <class W> double chargeT(W weight, bool express = false) { return double(weight) * 40 + (express ? 50 : 0); }
```

Time: overload resolution (and template instantiation) happens at compile time, so there is no run-time cost; the call itself is O(1) in every design. Space copied per call: the overloaded version copies a double (8 B) and two bool (2 B); the struct version passed by const reference copies only an address (8 B), although building the struct at the call site writes its fields. Handling a new option (for example a 'fragile' flag): in the overload design every overload needs a new trailing parameter, and the number of combinations grows, with new ambiguity risk; in the struct design one new field with a default value is added and existing callers do not change. Overloads give the shortest calls for common cases and type-based dispatch; the struct scales better with options.

# Q4 – Specifying a Class with an Array Inside It

## Savings Account Mini-Statement

### Class specification (account.cpp)

```
// Q4 - Savings Account Mini-Statement: class with a private array of 10 transactions
#include <iostream>
#include <string>

class SavingsAccount {
    static const int MAXTX = 10;
    struct Tx { char type; double amount; };
    int accNo;
    double balance;
    Tx tx[MAXTX];
    int count;
    void record(char t, double a);
public:
    SavingsAccount(int no, double bal) : accNo(no), balance(bal), count(0) {}
    double getBalance() const { return balance; }       // defined inside the class
    void deposit(double amt);                           // defined outside
    bool withdraw(double amt);                          // defined outside
    void miniStatement() const;                         // defined outside
};

void SavingsAccount::record(char t, double a)
{
    if (count == MAXTX) {                       // 11th transaction: shift left, discard oldest
        for (int i = 1; i < MAXTX; ++i) tx[i - 1] = tx[i];
        count = MAXTX - 1;
    }
    tx[count++] = {t, a};
}
void SavingsAccount::deposit(double amt)  { balance += amt; record('D', amt); }
bool SavingsAccount::withdraw(double amt)
{
    if (amt > balance) return false;
    balance -= amt; record('W', amt); return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini statement for account " << accNo << " (last " << count << ")\n";
    for (int i = 0; i < count; ++i)
        std::cout << "  " << (tx[i].type == 'D' ? "Deposit  " : "Withdraw ") << tx[i].amount << '\n';
    std::cout << "  Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount a(1001, 5000);
    for (int i = 1; i <= 12; ++i) {
        if (i % 3 == 0) a.withdraw(100 * i); else a.deposit(100 * i);
    }
    a.miniStatement();
    std::cout << "Overdraw attempt (50000): " << (a.withdraw(50000) ? "accepted" : "rejected") << '\n';
}
```

```
$ cd Q4
$ g++ -std=c++17 -Wall -Wextra account.cpp -o account && ./account
Mini statement for account 1001 (last 10)
  Withdraw 300
  Deposit  400
  Deposit  500
  Withdraw 600
  Deposit  700
  Deposit  800
  Withdraw 900
  Deposit  1000
  Deposit  1100
  Withdraw 1200
  Balance: 6800
Overdraw attempt (50000): rejected
```

Test: 12 transactions were made (deposits and every third one a withdrawal). The mini statement shows only the last 10 (transactions 3 to 12), the two oldest were discarded, and an overdraw attempt of 50000 was rejected. getBalance() is defined inside the class; deposit(), withdraw(), miniStatement() and record() are defined outside using SavingsAccount::.

**Inquiry**

- **Why the array is private:** the array and count must stay consistent with balance and with the 'last 10' rule. If outside code could write tx[] or count directly, it could falsify the statement or write beyond index 9. Access is only through deposit(), withdraw() and miniStatement(), which enforce the rules.
- **11th transaction – shifting:** copy tx[1..9] to tx[0..8] and put the new one at tx[9]; the data is always in order, so printing is a simple loop, but each add after the array is full costs 9 element copies, O(K).
- **11th transaction – circular index:** overwrite the oldest slot tx[start] and move start = (start + 1) % K; O(1) per add, but printing has to start at start and wrap around.

**Design Decision – last 50 transactions**

With the array design only the constant MAXTX changes from 10 to 50; the public interface (deposit, withdraw, miniStatement) and all callers stay the same, so that is the smallest change. With shifting the cost per add becomes about 49 copies, so I would change the storage to a circular index at the same time – it is still a change inside the private part only. Measured for 200,000 transactions at -O2: K = 10 → 1.7 ms, K = 50 → 2.7 ms (shifting).

**Alternative Concept and Complexity Analysis**

```
// std::deque version                    // circular buffer version
std::deque<Tx> h;                        Tx tx[K]; int start = 0, count = 0;
h.push_back(t);                          if (count < K) tx[(start+count++) % K] = t;
if (h.size() > K) h.pop_front();         else { tx[start] = t; start = (start+1) % K; }
```

| Design | Time to add | Space | Notes |
|---|---|---|---|
| Shifting array | O(K) once full (O(1) before) | K × 16 B + count | Simple, ordered, contiguous |
| Circular buffer | O(1) | K × 16 B + start + count | Needs modulo when printing |
| std::deque | O(1) amortised | ≈ K elements plus block/map overhead (heap) | Grows by blocks; extra allocation |

sizeof(Tx) is 16 bytes (char + padding + double), so K = 10 uses 160 bytes and K = 50 uses 800 bytes in the array designs.

# Q5 – Static Members and Arrays of Objects

## College Bus Seat Tickets

### Class design derived from the principle

- Data belonging to the whole class – the number of seats issued and the CAPACITY – is declared static: issued (static int) and CAPACITY (static const int). Per-ticket data (seat, name) stays non-static.
- A static data member is only declared inside the class; it must be defined exactly once outside, in one source file: int BusTicket::issued = 0;. Memory for it is created once, before main, however many objects exist.
- A static member function (count()) can be called without an object (BusTicket::count()), so it has no this pointer and therefore cannot use non-static members such as seat or name; it can use only static members.
- An array such as BusTicket trip[4] needs a constructor for each element. Because the constructor is explicit and takes a string, each element is initialised explicitly: {BusTicket("Asha"), ...}. Without an initialiser list a default constructor would be required.
- Count when tickets are created in a loop or function: the static counter belongs to the class, not to the object, so it keeps its value after local objects are destroyed. Nothing decrements it automatically.

### Predict the Output (Token)

```
#include <iostream>
class Token {
static int next;
int id;
public:
Token() { id = ++next; }
int get() const { return id; }
static int issued() { return next - 100; }
};
int Token::next = 100;
int main()
{
Token counter[3];
Token extra;
std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
<< ' ' << Token::issued() << '\n';
}
```

**My prediction (before running):** 101 103 104 4

```
$ cd Q5
$ g++ -std=c++17 -Wall -Wextra token.cpp -o token && ./token
101 103 104 4
```

Observed output equals the prediction. next starts at 100 because of its definition int Token::next = 100;. The constructor uses pre-increment, so the first object gets 101. The array of 3 runs the constructor 3 times (101, 102, 103), then extra runs it a 4th time (104). So counter[0] = 101, counter[2] = 103, extra = 104 and issued() = next − 100 = 4.

### Complete the Code (TODO 1 and TODO 2)

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                  // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
```

```
        // TODO 1
        if (issued < CAPACITY) { ++issued; seat = issued; }
    }
    static int count()
    {
        // TODO 2
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};
int BusTicket::issued = 0;                  // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

```
$ cd Q5
$ g++ -std=c++17 -Wall -Wextra bus.cpp -o bus && ./bus
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

The array holds 4 tickets while the capacity is 3, so the first three get seats 1–3 and Dev goes to the waiting list (seat stays 0). Issued: 3 – the waiting ticket does not increase the count.

**Tickets created inside a function called several times (bus_func.cpp)**

```
$ cd Q5
$ g++ -std=c++17 -Wall -Wextra bus_func.cpp -o busf && ./busf
Call 1:
Asha: Seat 1
Bala: Seat 2
Issued after call 1: 2
Call 2:
Chitra: Seat 3
Dev: Waiting
Issued after call 2: 3
Call 3:
Esha: Waiting
Farid: Waiting
Issued after call 3: 3
```

Each call creates two local tickets that are destroyed on return, but issued keeps its value: after call 1 it is 2, call 2 issues seat 3 and puts Dev on the waiting list (3), and call 3 puts both passengers on the waiting list because the bus is already full. The static count measures seats issued over the whole run, not the number of live objects.

**Alternative Concept and Complexity Analysis**

```
// Global counter                  // TicketOffice object passed to every ticket
int issued = 0;                    class TicketOffice { int issued = 0, cap; public: ... int next() { return issued < cap ? ++issued : 0; } };
BusTicket(name) { if (issued<3)...} BusTicket(const std::string& n, TicketOffice& office) : seat(office.next()), name(n) {}
```

| Design | Time / space | Encapsulation | Two buses at once |
|---|---|---|---|
| static data member | O(1) / one counter | Counter is private to the class | No – one shared counter for all tickets |

| Design | Time / space | Encapsulation | Two buses at once |
|---|---|---|---|
| global variable | O(1) / one counter | None – any code can change it | No |
| TicketOffice object | O(1) / one counter per office | Good – counter hidden in the office | Yes – one office per bus |

The static design is the simplest and fully encapsulated for a single bus; if the college runs several buses, the TicketOffice design is better because each bus has its own counter and the class has no hidden global state (also easier to unit-test).

# Q6 – Friend Functions and Inline Functions

## Train Journey Planner

### Complete the Code (TODO 1 and TODO 2)

```
#include <iostream>
class Time {
int h, m;
public:
Time(int hh, int mm) : h(hh), m(mm) {}
friend int minutesBetween(const Time& a, const Time& b);
friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
   // TODO 1
   int x = a.h * 60 + a.m, y = b.h * 60 + b.m;
   return x > y ? x - y : y - x;
}
bool earlier(const Time& a, const Time& b)
{
   // TODO 2
   return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main()
{
Time departure(9, 45), arrival(13, 10);
std::cout << minutesBetween(departure, arrival) << ' '
<< std::boolalpha << earlier(arrival, departure) << '\n';
}
```

```
$ cd Q6
$ g++ -std=c++17 -Wall -Wextra time.cpp -o time1 && ./time1
205 false
```

9:45 = 585 minutes, 13:10 = 790 minutes, difference 205. earlier(arrival, departure) asks whether 13:10 is before 9:45, which is false. Output matches the expected 205 false.

### Correctness check (tests.cpp)

```
$ cd Q6
$ g++ -std=c++17 -Wall -Wextra tests.cpp -o tests && ./tests
9:45 vs 13:10  -> 205 min, earlier(a,b)=true (hand: 205, true)
11:50 vs 12:15 -> 25 min, earlier(c,d)=true (hand: 25, true)
12:15 vs 11:50 -> 25 min, earlier(d,c)=false (hand: 25, false)
9:45 vs 9:45   -> 0 min, earlier(a,e)=false (hand: 0, false)
```

Hand verification: 9:45 → 13:10 = 3 h 25 min = 205; 11:50 → 12:15 (either side of noon) = 10 + 15 = 25 in both directions, so the function is symmetric (absolute difference); the same time twice = 0 and earlier is false (strictly earlier). All match.

**Design questions**

- **Why a friend:** the function treats two objects equally, but a member function would make one object special (a.minutesTo(b)). A friend is an ordinary non-member function that is granted access to the private h and m of both objects, so the call stays symmetric: minutesBetween(a, b).
- **Friend versus member:** a member is called with the dot operator on an object and has an implicit this; a friend is called like a normal function, has no this and receives both objects as parameters. A friend is not a member and is not inherited; it can access private members only because the class declares it.
- **Should they be inline?** inline is a request to the compiler to substitute the function body at the call site to avoid call overhead (and a permission to define it in headers without breaking the one-definition rule). The compiler may ignore it. For two-line functions called millions of times it is sensible, but the measurements below show that at -O2 the compiler already inlines such small functions.
- **If the members were public:** any code could set invalid values such as 27:99, and the internal representation (h, m) would be locked in; changing it to minutes-since-midnight would break every user. Private data plus controlled access keeps the invariant and the freedom to change.

**Growth Investigation – 10,000,000 calls (bench.cpp, best of 5 runs)**

```
$ cd Q6
$ g++ -std=c++17 -O0 bench.cpp -o bench0 && ./bench0
10,000,000 calls each, best of 5 (ms)
friend, inline     : 88.015
friend, no inline : 90.6117
member minutesTo   : 91.1036
public getters     : 115.34
```

```
$ cd Q6
$ g++ -std=c++17 -O2 bench.cpp -o bench2 && ./bench2
10,000,000 calls each, best of 5 (ms)
friend, inline     : 32.2322
friend, no inline : 30.4756
member minutesTo   : 30.1761
public getters     : 30.7374
```

| Version (ms for 10M calls) | -O0 | -O2 | Comment |
|---|---|---|---|
| friend, inline | 88.0 | 32.2 | keyword ignored at -O0 |
| friend, noinline attribute | 90.6 | 30.5 | forced out-of-line call |
| member minutesTo | 91.1 | 30.2 | same as friend |
| public getters | 115.3 | 30.7 | 4 extra calls at -O0 |

Explanation: at -O0 the compiler does no optimisation and does not honour inline, so inline and non-inline friend functions take the same time (88 vs 91 ms – within noise), and the getter version is about 30% slower because each call to getH() and getM() is a real function call. At -O2 all four versions take about 30 ms: the compiler inlines the small functions anyway, and the loop cost (the modulo operations used to create test times) dominates. About 3 ns per iteration, so the function call overhead is small even at 10 million calls. Timing values vary a little between runs.

**Alternative Concept and Complexity Analysis**

```
// member functions                    // public getters
int Time::minutesTo(const Time& o) const;     int getH() const; int getM() const;
bool Time::isEarlier(const Time& o) const;    // free functions use a.getH()*60 + a.getM()
```

Time per call is O(1) for the friend, member and getter designs (a few arithmetic operations) and is identical after inlining (≈30 ms for 10 M calls at -O2); at -O0 the getter design is slower because of the

extra calls. Encapsulation: friend – the class exposes nothing new publicly, but each friend is a permanent exception that must be listed inside the class; member – access is natural, but the call is asymmetric (a.minutesTo(b)); getters – the representation (hours and minutes) becomes part of the public interface and any external code may depend on it.

**Deployment Decision**

I would keep the friend functions for these two symmetric operations. Evidence: at -O2 there is no measurable speed difference between friend, member and getter versions, so the decision is based on design: friends keep the public interface small and do not leak the (h, m) representation, whereas getters make it part of the contract. If the set of operations grows, I would turn earlier() into a friend operator< and keep the representation private; a member version would be acceptable too because it is equally fast.

# Q7 – Diagnose, Redesign and Validate

## Selected program: Q4 – Savings Account Mini-Statement (shifting array)

### A. Diagnose the Limitation (2 marks)

The requirement changed in Q4 from the last 10 to the last 50 transactions. In the original design, once the array is full every deposit() or withdraw() shifts K − 1 elements (record() loops from i = 1 to MAXTX − 1). The cost per transaction therefore grows linearly with K: O(K). The cause is the design characteristic that the oldest item is always at index 0, so adding a new item moves all the others. Evidence from my measurement of 200,000 transactions at -O2: K = 10 → 1.7 ms, K = 50 → 2.7 ms, K = 500 → 12.0 ms, K = 5000 → 300 ms (about 175 times slower than K = 10), and at -O0 K = 5000 → 1094 ms.

### B. Define the Design Objective (2 marks)

Make the cost of adding a transaction independent of the history size K: at K = 5000 the redesigned add() must take less than 2 ms for 200,000 transactions (at least 100 times faster than the original 300 ms) and at every K it must stay within 2× of its K = 10 time, using only O(1) extra memory and without changing the public interface (deposit, withdraw, miniStatement).

### C. Design the Revised Solution (4 marks)

Change: keep the same private array of K transactions but add an index start that marks the oldest entry (circular buffer). C++ features used: the private data of the class, an int index and the modulo operator; no library container is needed. Pseudocode (the class interface does not change):

```
class SavingsAccount {
   Tx tx[K]; int start = 0, count = 0;          // extra memory: one int (start)
public:
   void record(Tx t) {
      if (count < K) tx[(start + count++) % K] = t;      // not full: append
      else { tx[start] = t; start = (start + 1) % K; }   // full: overwrite oldest
   }
   void miniStatement() const {
      for (int i = 0; i < count; ++i) print(tx[(start + i) % K]);  // oldest -> newest
   }
};
```

Trade-offs: add changes from O(K) to O(1); extra memory is one int, i.e. O(1); miniStatement() is still O(K) (it has to print K lines anyway) but needs a modulo per element; the array is no longer stored in chronological order in memory, so code that wants contiguous chronological data must copy it out first. A std::deque would also give O(1) but allocates heap memory in blocks; the circular array keeps the original fixed memory layout.

**D. Predict Before Implementing (1 mark)**

Prediction: the circular buffer's time for 200,000 adds will be roughly constant for every K (about 0.5–1 ms), because each add does a constant number of operations, while the shifting version will grow in proportion to K (K = 5000 about 100 times slower than K = 50). Expected improvement at K = 5000: more than 100×; at K = 10 only a small gain (about 1.5–2×), because only 9 elements were being shifted. Measure: milliseconds for 200,000 adds at -O2 and -O0.

**E. Implement and Validate the Redesign (2 marks)**

Same test for both versions (Q7/bench.cpp, templates ShiftHistory<K> and CircularHistory<K>, 200,000 add() calls, K = 10, 50, 500, 5000):

```
$ cd Q7
$ g++ -std=c++17 -O2 bench.cpp -o bench2 && ./bench2
200,000 transactions, time in ms (original shift | redesign circular)
K=10     : 1.712 | 0.732885
K=50     : 2.73116 | 0.70124
K=500    : 12.0136 | 0.790084
K=5000   : 300.271 | 0.686809
```

```
$ cd Q7
$ g++ -std=c++17 -O0 bench.cpp -o bench0 && ./bench0
200,000 transactions, time in ms (original shift | redesign circular)
K=10     : 3.9729 | 0.882292
K=50     : 12.4159 | 0.746566
K=500    : 121.472 | 0.750588
K=5000   : 1094.41 | 0.790245
```

| K | Original shift (ms) | Redesign circular (ms) | Speed-up (-O2 \| -O0) |
|---|---|---|---|
| 10 | 1.71 \| 3.97 | 0.73 \| 0.88 | 2.3× \| 4.5× |
| 50 | 2.73 \| 12.42 | 0.70 \| 0.75 | 3.9× \| 16.6× |
| 500 | 12.01 \| 121.47 | 0.79 \| 0.75 | 15.2× \| 161.8× |
| 5000 | 300.27 \| 1094.41 | 0.69 \| 0.79 | 437× \| 1385× |

(Each cell: -O2 value | -O0 value.)

**F. Defend the Final Design (2 marks)**

The objective was achieved. The original time grows from 1.7 ms to 300 ms (≈175×) as K goes from 10 to 5000, while the circular version stays between 0.69 and 0.79 ms, within 2× of its K = 10 time, and at K = 5000 it is about 437× faster (more than the required 100×). The prediction was correct on the shape (flat versus linear) and the K = 5000 gain; it slightly underestimated the gain at K = 10 at -O0 (4.5× instead of 1.5–2×) because the unoptimised shifting loop is slow. The new costs are one extra integer, a modulo operation per element when printing, and the loss of chronological order in memory. The original shifting array may still be preferable for small K (10) when the statement is printed very often and simplicity matters: it is easier to read and debug, stores items contiguously in order, and at K = 10 it costs only about 1.7 ms per 200,000 transactions in absolute terms. The cost of printing the mini statement (O(K) in both designs) was not measured here.